

Proyecto 1 – 2024 – Gestión de riesgos y excepciones en MIPS

Guerrero Serrano, Eduardo (NIP: 816106)

martes, 1 de abril de 2025

Breve resumen

El objetivo de este proyecto es aprender a trabajar con un lenguaje de descripción de hardware, usando un entorno de simulación para poder programar, compilar y ver cronogramas.

El proyecto se basa en el diseño y mejora de un procesador MIPS segmentado, este procesador está integrado en un sistema SOC y trabaja junto a IO_Data_Memory_subsystem, el cual abarca la DRAM de datos y algunos registros de entrada y salida mapeados en memoria.

Una de las grandes mejoras al procesador MIPS es la gestión de riesgo, basada en el uso de dos nuevos componentes, la Unidad de anticipación y la Unidad de detención. La Unidad de anticipación (UA) se encarga de controlar dos multiplexores ubicados en la entrada de la ALU, para poder usar datos de manera anticipada. Mientras que la Unidad de detención (UD) es la encargada de parar una instrucción o matarla en caso de que sea necesario.

Además, se han añadido tres nuevas instrucciones, jal y ret, las cuales ya han sido trabajadas en la práctica 3 y lw_inc (incremento de lectura automática) la cual guarda el valor de memoria en un registro, e incrementa el valor de memoria.

Otra mejora es el control de excepciones ubicado en la etapa de memoria.

Por último, Este procesador tiene agregado una serie de contadores, para poder monitorizar diferentes parámetros.

Verificación de los requisitos funcionales:

RF1 JAL y RET

Descripción: Incluir las instrucciones JAL y RET ya trabajadas en prácticas en el MIPS del proyecto

¿Cómo? Para incluir las nuevas instrucciones JAL y RET he tenido que agregar una serie de señales al MIPS segmentado.

JAL: en el caso de la instrucción JAL ($BR(rt) \leftarrow PC+4$; $PC \leftarrow PC+4+4*SigExt(inm16)$) la unidad de control habilita una señal de jal que indica que la instrucción es una jal, esta señal hace que el salto sea tomado y va pasando por las distintas etapas hasta llegar a la etapa WB, en esta etapa en caso de que la señal esta activada el busW valdrá PC+4. Para que esto funcione también hemos tenido que guardar la señal de PC+4 e ir pasando la por las diferentes etapas hasta el WB.

RET: para la instrucción RET ($PC \leftarrow BR(rs)$) la unidad de control habilita una señal de ret, la cual sirve de condicionante para seleccionar la dirección de la siguiente instrucción (PC_in) si la señal RET esta activada la siguiente instrucción que se ejecute será la que este en el busA.

AOC2 - EINA - Universidad de Zaragoza

Para la unida de detención hay que tener en cuenta que si hay una instrucción RET, pero el registro RS esta en uso, la instrucción se tendrá que detener en la etapa ID hasta que el registro este disponible. Y para el caso de JAL la instrucción se detendrá si una instrucción desea leer el registro donde se escribe JAL en los dos siguientes ciclos.

Verificación: Para comprobar su correcto funcionamiento he pasado el código de la practica 3. En el proyecto esta como el Test 6. He utilizado una memoria de datos diferente, esta comentada dentro del archivo ram_128_32_p1_2025.vhd.

Su funcionamiento es:

Con la memoria de RAM con los siguientes valores (base 10):

@7: 800000

@8: 10000

@9: 6000

@10: 100

@11: 0

@12: 6

El programa debe acceder a estas direcciones de memoria usando un bucle y acumular sus valores. Por último escribir el resultado en la dirección @0

Etiqueta	Dirección	Instrucción (hexadecimal)	Instrucción	Descripción
Reset	@0x0	10210003	Beq R1, R1, INI	Salto a INI (@16)
IRQ	@0x4	1021003E	Beq R1, R1, RTI	Salto a RTI (@64*4)
DAtbort	@0x8	1021005D	Beq R1, R1, RT_Abort	Salto a RT_Abort (@96*4)
UNDEF	@0xC	1021006C	Beq R1, R1, RT_UNDEF	Salto a RT_UNDEF (@112*4)
INI	@0x10	08010004	Lw R1, x4(R0)	R1 = Mem(4) = 28
		08020008	Lw R2, x8(R0)	R2 = Mem(8) = 48
		0803000C	Lw R3, xC(R0)	R3 = Mem(12) = 4
		14040000	Jal R4, 0	R4 = @0x20; salto a la siguiente instrucción
		08250000	Lw R5, x0(R1)	R5 = Mem(R1) = 800000, 10000, 6000, 100, 0, 6
		04C53000	Add R6, R6, R5	R6 = R6 + R5
		1022000	Beq R1, R2, FIN	Si R1 = R2, salto a FIN
		04230800	Add R1, R1, R3	R1 = R1 + R3
		18800000	Ret R4	Salto a R4 (@0x20)
FIN		0C060000	Sw R6, x0(R0)	Mem(0) = R6
		1400FFFF	Jal R0, -1	Bucle infinito
RTI	@0x100	0FE10034	Sw R1, x34(R31)	Mem(52) = R1 guardar R1
		0BE10038	Lw R1, x38(R31)	R1 = Mem(56) = 1
		0FE17008	Sw R1, x7008(R31)	INT_ACK <= 1
		0BE10034	Lw R1, x34(R31)	R1 = Mem(52) recuperar R1
		20000000	RTE	Volver de la interrupción
RT_Abort	@0x180	1000FFFF	Beq R0, R0, -1	Bucle infinito

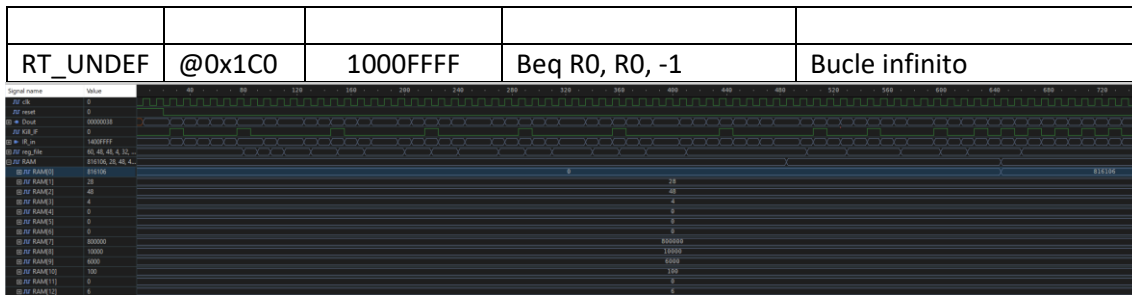
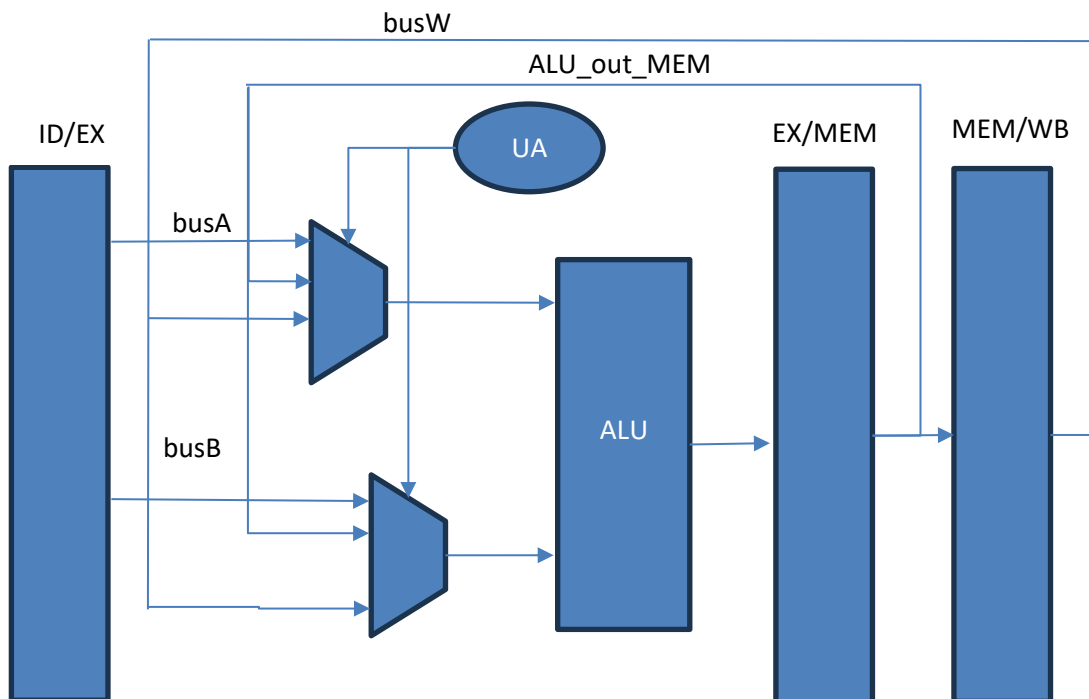


Imagen de Test6.awc

RF2 Anticipación de operandos

Descripción: El procesador es capaz de anticipar los operandos para las instrucciones LW, SW y ARIT a distancia 1 (siempre que el dato a anticipar no venga de un lw) y 2.

¿Cómo? La unidad de anticipación controla 2 MUX 4x1 ubicados en la entrada de la ALU, estos MUX tienen como entrada: el bus con el registro correspondiente, la salida de la ALU de la instrucción anterior y el busW de dos instrucciones posteriores.



En la imagen se muestran las señales que entran a los MUX relacionados con la UA

Dentro de la unidad de anticipación se encuentra como funciona el control del MUX. Para utilizar la señal de anticipación de la etapa MEM tiene que ser una instrucción válida en esta etapa y compara el registro de la instrucción con el registro que se va a escribir en MEM (RW_MEM) y comprueba la señal RegWrite_MEM esta activada. En el caso de utilizar la señal anticipada de la etapa WB tiene que ser una instrucción válida en esta etapa y compara el registro de la instrucción con el registro que se va a escribir en WB (RW_WB) y comprueba la señal RegWrite_WB esta activada.

El funcionamiento del segundo control del MUX es similar, pero coge el otro registro de la instrucción.

Verificación:

- En el banco de pruebas test_IRQ se realizan las siguientes anticipaciones:
 - De LW R1, 8(R0) a ADD r31, R1, R31: anticipación de Rs distancia 2 (tras detención de un ciclo del ADD)
 - De SUB r31, R31, R1 a LW R1, 0(R31): anticipación de Rs distancia 1.
 - De ADD R2, R1, R2 a SW R2, 7008(R6)
- Para comprobar el correcto funcionamiento de la anticipación de operandos he realizado el Test 8:

Este test realiza instrucciones de LW, SW y ARIT de forma consecutiva para comprobar que adquiere operandos de forma anticipada

Etiqueta	Dirección	Instrucción (hexadecimal)	Instrucción	Descripción
Reset	@0x0	10210003	Beq R1, R1, INI	Salto a INI (@16)
IRQ	@0x4	1021003E	Beq R1, R1, RTI	Salto a RTI (@64*4)
DAtbort	@0x8	1021005D	Beq R1, R1, RT_Abort	Salto a RT_Abort (@96*4)
UNDEF	@0xC	1021006C	Beq R1, R1, RT_UNDEF	Salto a RT_UNDEF (@112*4)
INI	@0x10	08010004	Lw R1, x4(R0)	R1 = Mem(4) = 1
		08020004	Lw R2, x4(R0)	R2 = Mem(4) = 1
		04221000	Add R2, R1, R2	R2 = R1+R2 = 2 (tarda un ciclo por lw uso) (usa R2 de forma anticipada en WB)
		0C02000C	Sw R2, xC(R0)	Mem(12) = R2 = 2 (usa R2 de forma anticipada en MEM)
		04221001	Sub R2, R1, R2	R2 = R1-R2 = -1 (usa R2 de forma anticipada en WB)
		08020004	Lw R2, x4(R0)	R2 = Mem(4) = 1 (usa R2 de forma anticipada en MEM)
		04221002	And R2, R1, R2	R2 = R1 AND R2 = 1 (tarda un ciclo por lw uso) (usa R2 de forma anticipada en WB)
		0C02000C	Sw R2, xC(R0)	Mem(12) = R2 = 1 (usa R2 de forma anticipada en MEM)
		1000FFFF	Beq R0, R0, -1	Bucle infinito
RTI	@0x100	0FE10020	Sw R1, x20(R31)	Mem(32) = R1 guardar R1
		0BE10024	Lw R1, x24(R31)	R1 = Mem(36) = 1
		0FE17008	Sw R1, x7008(R31)	INT_ACK <= 1

		0BE10020	Lw R1, x34(R31)	R1 = Mem(32) recuperar R1
		20000000	RTE	Volver de la interrupción
RT_Abort	@0x180	1000FFFF	Beq R0, R0, -1	Bucle infinito
RT_UNDEF	@0x1C0	1000FFFF	Beq R0, R0, -1	Bucle infinito

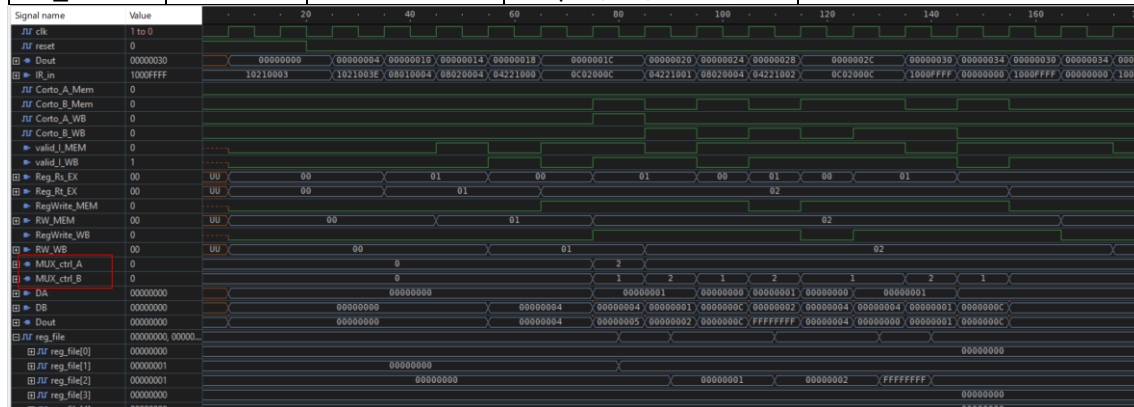


Imagen de Test8.awc

RF3 Riesgos de datos

Descripción: El procesador es capaz de detener la ejecución para evitar los riesgos causados por las dependencias en instrucciones lw-uso (distancia 1), beq o WRO (distancias 1 o 2).

¿Cómo? El procesador puede detener la ejecución de una instrucción con la señal stall_ID, esta señal se controla desde la unidad de detención.

Hay varios casos en los que hay un riesgo de datos y estos operandos no se pueden adquirir de ninguna forma, por tanto, hay que para la instrucción:

- **Lw_uso:** El load uso ocurre cuando la instrucción en EX es una carga y la instrucción necesita el dato de ese load, es decir MemRead_EX esta activada y el registro de la instrucción con el registro que se va a escribir en EX. Además, se tiene que cumplir que tanto la instrucción en EX, como la de ID, sean validas y que la instrucción quiera leer ese registro.
Esto es aplicable a los dos registros.
- **Beq:** Este riesgo ocurre cuando la instrucción es un BEQ y alguno de los registros es dependiente en alguna de las dos instrucciones anteriores.
Esto es aplicable a los dos registros.
- **Ret:** Este riesgo ocurre cuando la instrucción es un RET y alguno de los registros es dependiente en alguna de las dos instrucciones anteriores.
Es similar al Beq pero este solo es aplicable al registro en rs.
- **Jal_uso:** Este riesgo puede ocurrir cuando la instrucción en EX es un JAL valido y el registros que se va a usar en ID es el mismo que el que va a escribir la instrucción JAL o cuando la instrucción en MEM es un JAL valido y el registro que se va a usar en ID es el

mismo que el que va a escribir la instrucción JAL. La instrucción en ID tiene que ser valida y debe querer leer el registro.
Esto es aplicable a los dos registros.

Verificación:

- En el banco de pruebas test_IRQ se realizan las siguientes detenciones:
 - Test 1: 1 ciclo de detención por dependencia a distancia 2 entre ADD R1, R1, R1 y beq R1, R1, main.
 - Test 2: 1 ciclo de detención por dependencia lw-uso en varios casos. Por ejemplo: de LW R1, 8(R0) a SW r1, 7004(r6)
 - Test 3: No se para por riesgo de datos entre las instrucciones SW r1, 7004(r6) y ADD R1, R1, R1.
- Para comprobar su correcto funcionamiento he pasado el código del Test 9:

Este test simula los casos en los que puede haber un riesgo de datos. LW_uso, BEQ, RET, y JAL_uso

Etiqueta	Dirección	Instrucción (hexadecimal)	Instrucción	Descripción
Reset	@0x0	10210003	Beq R1, R1, INI	Salto a INI (@16)
IRQ	@0x4	1021003E	Beq R1, R1, RTI	Salto a RTI (@64*4)
DAtbort	@0x8	1021005D	Beq R1, R1, RT_Abort	Salto a RT_Abort (@96*4)
UNDEF	@0xC	1021006C	Beq R1, R1, RT_UNDEF	Salto a RT_UNDEF (@112*4)
INI	@0x10	08020010	Lw R2, x10(R0)	R2 = Mem(16) = 12
		08010004	Lw R1, x4(R0)	R1 = Mem(4) = 1
		04210800	Add R1, R1, R1	R1 = R1+R1 = 2 (Tarda 1 ciclo más por lw uso)
		10220003	Beq R1, R2, Fin	Si R1 = R2 hace un salto a Fin (Tarda 2 ciclos más por riesgo de datos beq)
		14040001	Jal R4, 1	R4 = @0x24; Saltar 1 instrucción
		08010000	Lw R1, x0(R0)	R1 = Mem(0) = 256
		04822000	Add R4, R4, R2	R4 = R4+R2 = @0x30 (Tarda 1 ciclo más por riesgo de datos de JAL_uso)

		18800000	Ret R4	Salto a R4 (@0x30) (Tarda 2 ciclos más por riesgo de datos ret)
Fin		1000FFFF	Beq R0, R0, -1	Bucle infinito
RTI	@0x100	0FE10020	Sw R1, x20(R31)	Mem(32) = R1 guardar R1
		0BE10024	Lw R1, x24(R31)	R1 = Mem(36) = 1
		0FE17008	Sw R1, x7008(R31)	INT_ACK <= 1
		0BE10020	Lw R1, x34(R31)	R1 = Mem(32) recuperar R1
		20000000	RTE	Volver de la interrupción
RT_Abort	@0x180	1000FFFF	Beq R0, R0, -1	Bucle infinito
RT_UNDEF	@0x1C0	1000FFFF	Beq R0, R0, -1	Bucle infinito



Imagen de Test9.awc

RF4 Riesgos de control

Descripción: El procesador es capaz eliminar los riesgos de control causados por las instrucciones de salto: BEQ, JAL, RET y RTE.

¿Cómo? El procesador puede matar la ejecución de una instrucción con la señal kill_IF, esta señal se controla desde la unidad de detención.

Hay varios casos en los que hay un riesgo de datos y estos operandos no se pueden adquirir de ninguna forma, por tanto, hay que para la instrucción:

En el caso de que se ejecute la instrucción JAL, RET, RTE se matara la instrucción del estado IF, siempre y cuando la instrucción sea valida y no haya riesgos de datos.

Para el caso de la instrucción BEQ, hay que asegurarse de que se toma el salto antes de matar la instrucción IF.

Verificación:

- En el banco de pruebas test_IRQ se realizan varios saltos tomados en los que sólo se ejecutan las instrucciones adecuadas:
 - beq R1, R1, main
 - rte
- verificar un ejemplo en el que no se salte.
- Verificar jal, ret

Estas dos verificaciones están englobadas en el Test 10, el cual ejecuta un BEQ que no salta y una secuencia JAL, RET

Etiqueta	Dirección	Instrucción (hexadecimal)	Instrucción	Descripción
Reset	@0x0	10210003	Beq R1, R1, INI	Salto a INI (@16)
IRQ	@0x4	1021003E	Beq R1, R1, RTI	Salto a RTI (@64*4)
DAtbort	@0x8	1021005D	Beq R1, R1, RT_Abort	Salto a RT_Abort (@96*4)
UNDEF	@0xC	1021006C	Beq R1, R1, RT_UNDEF	Salto a RT_UNDEF (@112*4)
INI	@0x10	08010004	Lw R1, x4(R0)	R1 = Mem(4) = 1
		14040001	Jal R4, 1	R4 = @0x18; Saltar 1 instrucción (Mata la siguiente instrucción)
		08020004	Lw R2, x4(R0)	R2 = Mem(4) = 1
		10220001	Beq R1, R2, Fin	Si R1 = R2 hace un salto a Fin (En caso de que R1=R2, mata la siguiente instrucción, sino no)

		18800000	Ret R4	Salto a R4 (@0x18) (Mata la siguiente instrucción)
Fin		1000FFFF	Beq R0, R0, -1	Bucle infinito
RTI	@0x100	0FE10020	Sw R1, x20(R31)	Mem(32) = R1 guardar R1
		0BE10024	Lw R1, x24(R31)	R1 = Mem(36) = 1
		0FE17008	Sw R1, x7008(R31)	INT_ACK <= 1
		0BE10020	Lw R1, x34(R31)	R1 = Mem(32) recuperar R1
		20000000	RTE	Volver de la interrupción
RT_Abort	@0x180	1000FFFF	Beq R0, R0, -1	Bucle infinito
RT_UNDEF	@0x1C0	1000FFFF	Beq R0, R0, -1	Bucle infinito

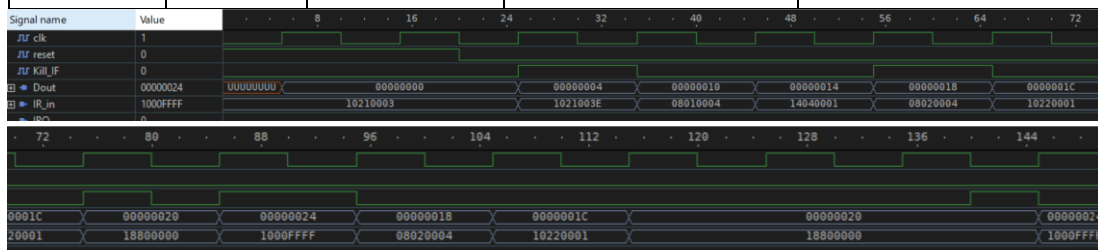


Imagen de Test10.awc

RF5 cambio a modo excepción

Descripción: Si el procesador está en modo usuario y recibe IRQ, ABORT o UNDEF pasa al modo correspondiente y ejecuta la rutina que indica la tabla de vectores de excepción. Si está en modo excepción ignora las señales hasta volver a modo usuario.

¿Cómo? El procesador envía las tres señales (IRQ, ABORT, UNDEF) al gestor de excepciones para que acepte o no la excepción.

La excepción se acepta si llega una de esas señales, las excepciones están habilitadas y el procesador no está parado.

El estado se actualiza si hay una excepción aceptada o se ejecuta una instrucción RTE, cuando el MIPS no está parado.

Verificación:

- IRQ: banco de pruebas test_IRQ en el que se realizan diversas IRQs. Todas ellas se atienden si y sólo si el procesador está en modo usuario. Se ejecuta una RTI que contabiliza el número de IRQs. Se verifica su funcionamiento correcto.

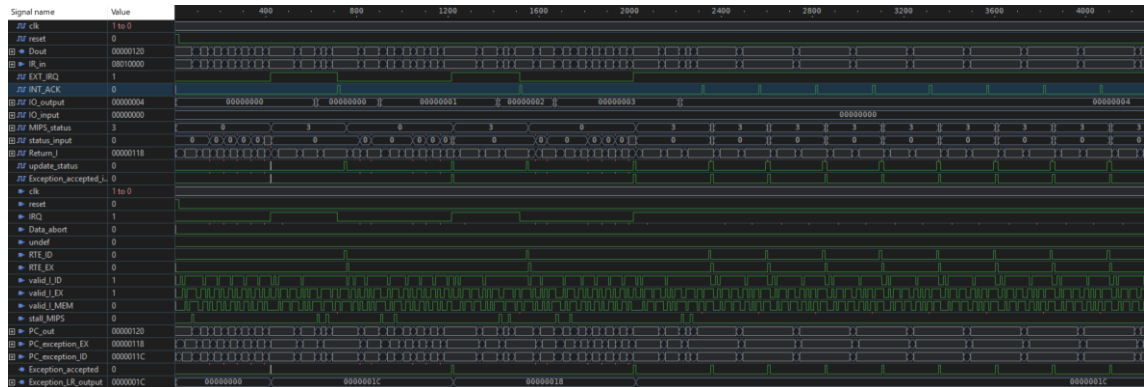


Imagen de testbech2_IRQ.awc

- Data Abort: en la memoria de instrucciones se incluyen dos bancos de pruebas en el que se realizan un acceso no alineado, y un acceso a una dirección fuera de rango. En ambos casos la ejecución salta a la rutina Data Abort (bucle infinito).

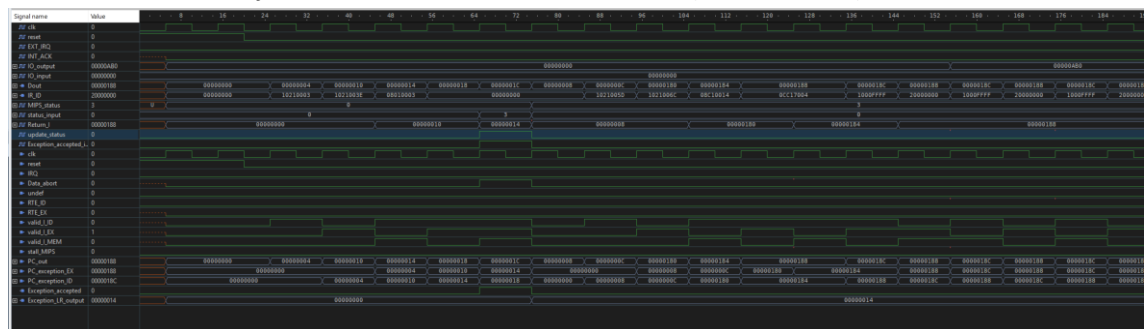


Imagen de testbech3_DataAbort.awc

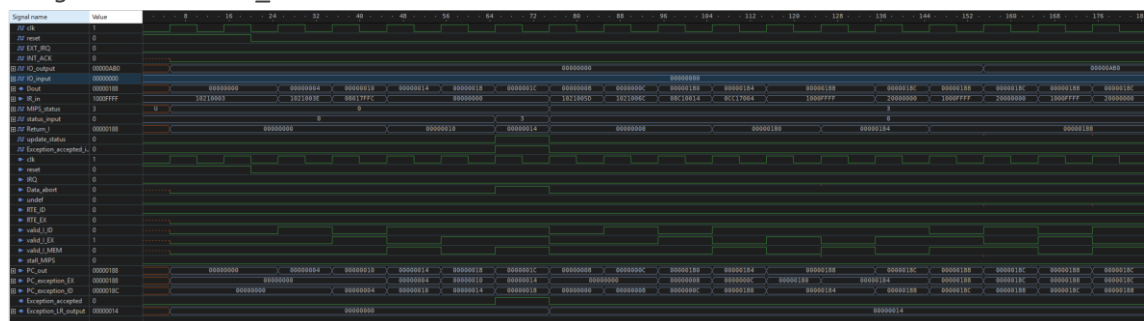


Imagen de testbech4_DataAbort.awc

- Undef: en la memoria de instrucciones se incluye un banco de pruebas en el que se ejecuta una instrucción con un código de instrucción desconocido. La ejecución salta a la rutina UNDEF (bucle infinito).

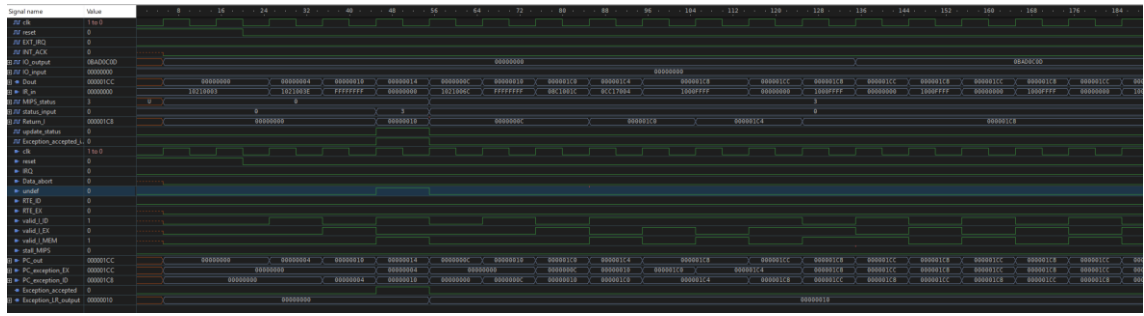


Imagen de testbech5_Undef.awc

RF6 retorno a modo usuario RTE

Descripción: Al terminar la excepción se vuelve a la ejecución original con la instrucción RTE

¿**Cómo**? Cuando se acepta una excepción se guarda en el `Exception_LR_output` la dirección de la instrucción que tocaba ejecutar antes de la excepción. Si se envía una ejecución RTE, el PC retorna a `Exception_LR_output`.

Verificación: Sólo tenéis que verificarlo para las IRQs. El código debe funcionar aunque se interrumpa repetidas veces. Para el resto de excepciones se asume que es un fallo irresoluble y se debe resetear el sistema, pero la gestión es idéntica que para las IRQs. En el banco de pruebas test_IRQ se puede comprobar que se retorna a la ejecución sin alterarla siempre que se siga un esquema de prólogo y epílogo apropiado (guardando y restaurando los registros en pila). Para ello se utiliza el registro 31 como SP.

RF7 contadores

Descripción: los contadores nos dan información de la ejecución del código

¿**Cómo**? Para cada controlador hay una señal que habilita el contador.

- `cont_I` cuenta el número de instrucciones ejecutadas.
Se habilita cuando la señal `valid_I_WB` está activa y no se ha parado el MIPS
- `cont_Exception` cuenta el número de excepciones aceptadas.
Se habilita cuando la señal `exception_accepted` está activa.
- `cont_data_stalls` cuenta el numero de paradas por datos.
Se habilita cuando la señal `stall_ID` esta activa y el contador de paradas por memoria no este habilitado.
- `cont_control_stalls` cuenta el número de paradas por control.
Se habilita cuando la señal `kill_IF` esta activa y el contador de paradas por memoria y contador de paradas por datos no esten habilitados.
- `cont_memory_stalls` cuenta el número de paradas por memoria.
Se habilita cuando la seña `IO MEM ready` no está activada.

Verificación: En el banco de pruebas test IRQ se ha comprobado que:

- Por cada instrucción válida que hace su etapa WB se incrementa el contador de instrucciones Ins. Si la instrucción no es válida no se incrementa.
- Data_stalls contabilizan bien los ciclos debidos a los riesgos de datos incrementando el número indicado en los test mencionados en el RF5.
- Control_stalls contabilizan bien los ciclos debidos a los riesgos de control, incrementando un ciclo en cada salto tomado (ver pruebas en el RF6).
- Exception_accepted contabiliza las excepciones aceptadas. Su cuenta coincide con la cuenta que realiza el propio código.
- Memory_stalls: cada vez que se pide una operación a memoria, y esta no esta “ready” se incrementa este contador.
- IMPORTANTE: las paradas deben incrementar sólo un contador. Si hay varios motivos, se cuenta el de mayor jerarquía : memory stall> data_stall> control_stall. Ya que el otro se mantendrá y se contabilizará cuando toque.

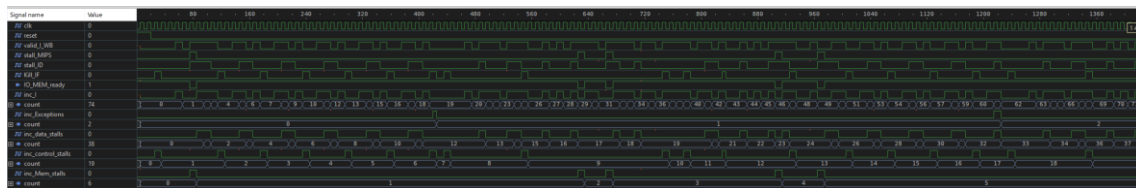


Imagen de testbech2_Counts.awc

Cuantificación de horas dedicadas

Indicar horas por tarea (y persona en el caso de parejas)

- Estudio del MIPS, VHDL, entorno, instalación....: 4h
- Adición de las nuevas instrucciones: 5h
- Gestión de excepciones: 2h
- Gestión de riesgos: 6h
- Depuración, verificación y programas de prueba: 18h
- Memoria: 5h

Conclusiones y Autoevaluación

Me ha resultado un trabajo muy útil e interesante. Durante la asignatura hemos ido adquiriendo conocimientos poco a poco, los cuales, gracias a la evaluación continua hemos trabajado en problemas y prácticas, el proyecto ha sido una unión de todos estos conocimientos. Aunque me ha parecido un proyecto duro en el que he tenido que dedicarle muchas horas, esto a supuesto que aumente mis conocimientos sobre la materia. Por lo que me ha gustado.

Por mi parte, creo que he adquirido los conocimientos sobre el procesador MIPS segmentado y su integración en el SOC. Estoy satisfecho con el trabajo realizado, aunque, me hubiese gustado haber llegado a tiempo para la entrega.

La calificación que me pongo es de un 8, considero que he realizado un buen trabajo y lo he podido probar con buenos test, pero siempre puede fallar algún imprevisto que no tengo en mente.

Agradecimientos

Aunque he hecho el proyecto completamente solo, me gustaría darle las gracias a Daniel Ubalde. Ha sido un compañero con el he podido discutir varios temas del proyecto y me ha dado puntos de vista que no se me habían ocurrido.